How We Scaled API Performance in a High-Load System and Avoided CPU Overload

CLOUD API

July 21, 2022

Share this article

In cloud solutions, an API plays the role of a gateway that manages incoming requests from all entities: developers, end users, third-party solutions, etc. But each API has a limited number of requests it can process. And when an API is overloaded, your whole system essentially stops, as it can't output any results without processing requests first.

load on your

How can you scaling?

How clou

rticle, we share our experience scaling a highly loaded API that processes resource-intensive requests. We also show how to calculate the future load on the system and choose the most suitable instances for scaling, as well as our scaling scheme and debugging activities.

This article will be helpful for cloud development teams that want to ensure their projects don't have API performance issues.

What type of scaling should you choose for your system?

When developers face API overload caused by too many requests, the most obvious thing to do is allocate as many processing resources to APIs as possible. However, this method of performance scaling comes at a high cost for your project.

The pricing of almost any cloud service depends on the number of computing resources you rent, so the more you allocate for APIs, the higher your project budget will need to be. That's why it's best to look for a balance between system performance and required computing resources.

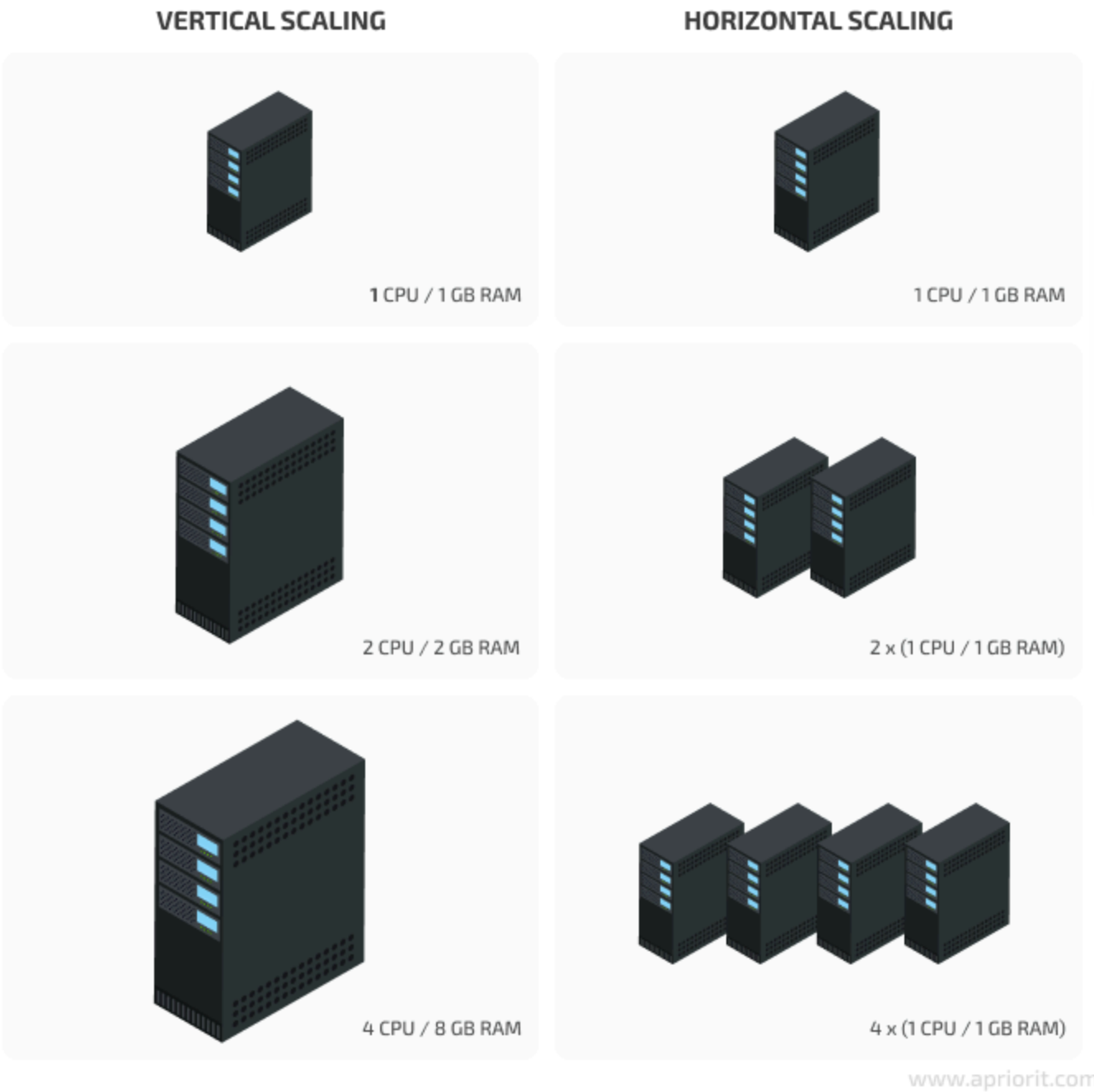
There are two key options for scaling cloud computing resources:

- **Vertical scaling**, or **scaling up or down**, means adding more resources to an instance you already have by adding more virtual machines, changing database performance levels, etc. Vertical scaling allows you to enhance the performance of your service when there's high demand and scale down when the demand subsides. It's mostly used for one-time events and ad-hoc fixes.

Contents:

- **Horizontal scaling**, or **scaling in or out**, means adding more instances to your current ones in order to divide the load between several endpoints. To scale horizontally, you need to enhance your system with additional physical computing power, create sharded databases, or deploy more nodes. Scaling out is a good way to improve your system's performance for long periods of time or even permanently.

Contents:



Have a questio

Ask our expe

Lidiia
R&D

Contents:

Conclusion

Contents:

The type of scaling you choose should depend on the requirements for your system and the issues you need to solve by scaling.

Let's examine how to scale cloud resources to improve API performance in a real-life project. For one of our recent projects, we created a system prototype with the following characteristics:

- Each request takes between 30 seconds and one minute and up to 20% of server CPU power.
- An excessive number of requests should not cause the machine to freeze and make it inoperable.

On top of that, our client wanted the system to:

- Be able to process up to five million requests per day
- Withstand a heavy bandwidth load in order to communicate with third-party services

We decided to deploy this system in AWS, as it provides high-capacity services with end-to-end scaling solutions. For our project, we selected horizontal scaling for the following reasons:

- We needed to be able to deploy the system in different availability zones to ensure fault tolerance.
- The application needed to be able to process up to five million requests a day, and vertical scaling would not withstand such a heavy load.
- If the server encountered issues, it could redirect requests to another instance.

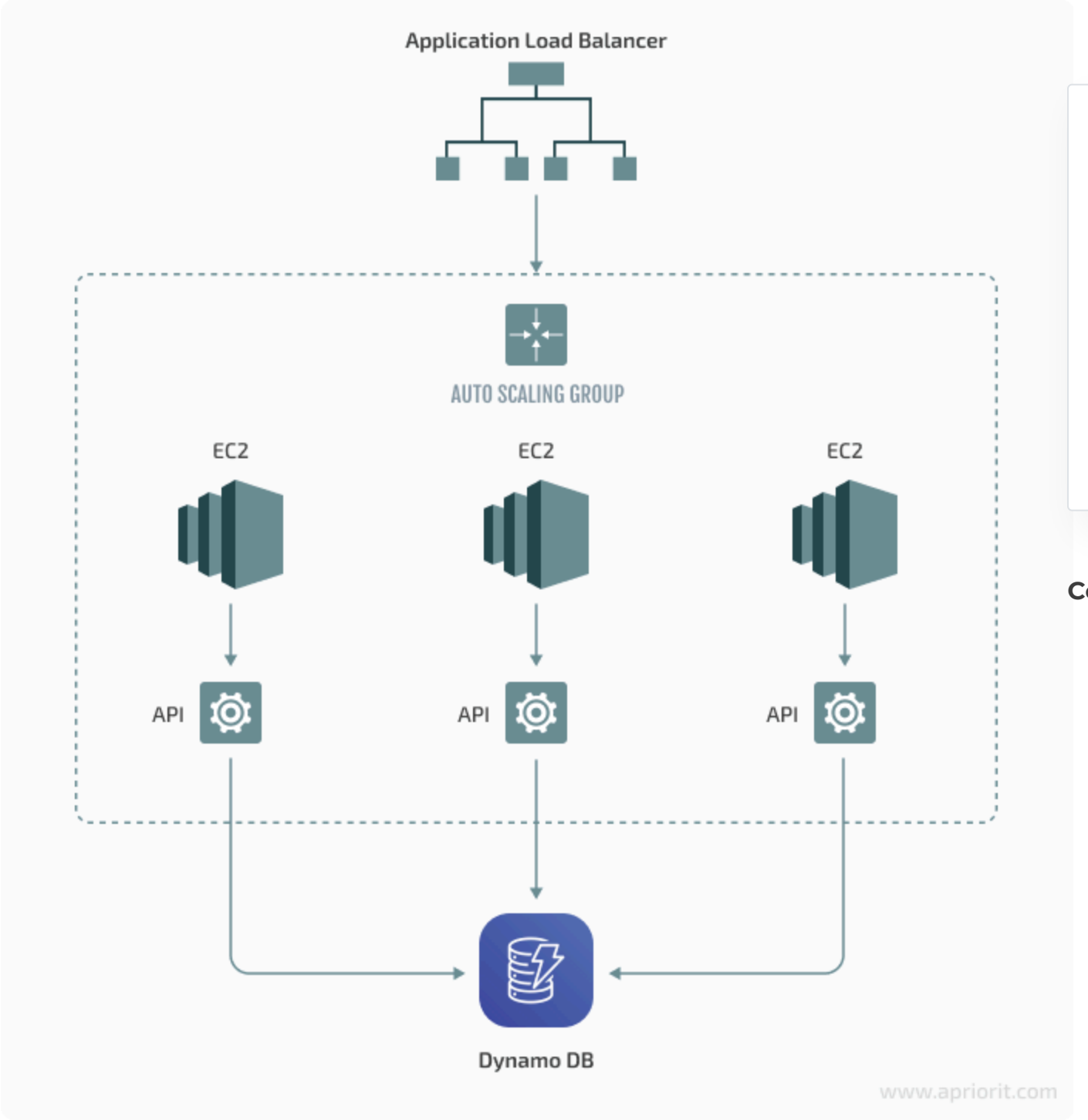
Here's how we balanced the load on the system and scaled it with AWS resources:

Have a questio

Ask our expe

Lidiia
R&D

Contents:



Have a questio

Ask our expe

Lidiia
R&D

Contents:

Our customer's project had an additional issue we needed to address when planning the cloud environment: Each API request to the system put a high load on the CPUs of EC2 machines. We quickly reached 100% CPU load, and the system froze.

To solve this and many other issues, we started looking for the most efficient way of scaling API performance in a high-load AWS project.

RELATED PROJECT

Building AWS-based Blockchain Infrastructure for International Banking

Discover how our client received the AWS infrastructure that satisfied their requirements and met all the project deadlines. Find out how Apriorit experts helped the client to conduct a project demo for several international banks and government organizations.

Have a questio

Ask our expe

Lidii
R&D

Contents:

Project details

Conclusion

How can you choose the features for your system?

To answer this question, you need to know the future load on your future. You can do it by forecasting the possible load or analyzing customers' requirements. For this project, we were lucky because the customer knew the approximate future load on the platform, so we could start designing a scaling scheme right away.

When a customer doesn't know the estimated load, we assess the project and provide several scaling options depending on our experience with previous projects.

Here's an example of our proposal for a system that processes 3, 30, and 300 requests per second. This comparison helps the customer choose between an affordable cloud environment and rich scalability options. We used the AWS Pricing Calculator to show the cost of each option.

Feature	3 req/sec	30 req/sec	300 req/sec

Contents:

REST API	SQS Lambda DynamoDB	Application Load Balancer EC2 DynamoDB	Application Load Balancer EC2 DynamoDB
AWS price	$246 / month	$2,920 / month	$25,682 / month
Integration time (DevOps time)	40 hours	80 hours	100 hours

Our calculations include the option of a gradual load increase and changes to the system depending on the load. Also, the customer can choose the three requests per second option for a prototype and then add scaling and debugging to it.

READ ALSO

Have a questio

Ask our expe

Lidii
R&D

Contents:

How can you choose instances for scaling?

The correct choice of instances for a project helps you rent the right amount of cloud computing resources to work under a certain load. It saves your project from crashes and budget bloat. But the only way to know for sure which instances you need is to build a prototype and conduct load testing. That's exactly what we did for our project:

1. Wrote a simple application prototype with unchangeable logic.
2. Launched the easiest and cheapest instance on AWS and loaded it until it failed.
3. Saw how many queries this instance completed and which criteria made it fail.
4. Launched a more powerful instance and tried to crash it with our prototype.

Usually, an instance crashes because of CPU or RAM overload, traffic limits, or third-party service limits. We used JMeter to find the maximum load on AWS instances until we found the most suitable ratio of instance parameters and time to crash.

Knowing the approximate minimum load on the project and the capacity of one machine, we could choose an optimal instance. For example, during development and testing of our customer's system, up to 10 people worked with it simultaneously. Usually, you only need to keep one machine running for domain development.

Load testing helped us find out that for our customer's system, the most suitable instance configuration was 4 CPUs, 8 GB RAM, unlimited traffic, and no third-party services limits. Such an instance can process 60 to 120 requests per minute, or 2 to 3 requests per second, before crashing.

After testing the prototype, we could predict how many simultaneous requests one instance could handle and how many instances the project would need. Based on this information, we offered our customer the optimal number of instances, determined their optimal configuration, and calculated their cost.

Contents:

RELATED PROJECT

Building a Microservices SaaS Solution for Property Management

Explore the real-life success story of implementing an updated microservices-based SaaS platform that offers better scalability, easier code maintenance, and a more enjoyable user experience.

Project details

How can you design and debug a cloud scaling system?

Unfortunately, you may not understand why a new system crashes in production. But crashes may happen if development teams neglect load testing.

When we tell a client that hosting an application in a scalable cloud service will solve performance issues, we mean that we've already tested the system under a heavy load.

When you work on a new system, creating a prototype helps you plan and estimate the project, but it won't solve all API performance issues. Connecting a prototype to the application load balancer is not enough, since problems start when you run several instances.

For our customer's project, we managed to debug the following issues with scaling system performance:

Have a questio

Ask our expe

Lidiia
R&D

Contents:

1. **Launching a new instance took five to seven minutes.** During this time, the load could increase by twofold or threefold, crashing the instance upon launch. We managed to launch a new instance in three minutes by reducing the number of scripts started when an endpoint launches. We did it by creating a custom AMI image for AWS EC2 instance. The new AMI already has all libraries and we don't need extra time to launch it.
   The servers worked in the alarm zone for about one minute. After the alarm about it, we launch a new instance within two minutes.

2. **Failure of one API request can affect other requests.** Our system frequently used browsers for its work, which have their own performance limitations. When one request overloaded a browser, it couldn't process subsequent requests. To fix this, we isolated each process and limited the load by creating internal requests and a request queue for each server.
   The first step was to create a worker that would run our program independently so that in case of errors, we would lose only one request instead of all operations. This worker takes one task at a time from the queue after completing the previous one. In our case, the CPU load per worker was 7%.

   Next, we increased the number of workers until the CPU was 80% loaded. This way, we got an isolated process with a predictable load and created a number of parallel tasks for the instance to process.

3. **Load balancer evenly distributes the load between instances by default.** Because of this, instances that launch first have a two to three times higher load than others, leading to a higher chance of crashing. We made the application load balancer send new requests to instances with the fewest active connections until all instances had an even load.

4. **Our API keeps the connection active until it gets a response.** Under a heavy load, it took our system three to five minutes to respond, and during this time the application load balancer ended the connection. We increased the maximum

Contents:

connection time for the API to three to four minutes. During this time, we can create a new instance if needed.

This approach allowed us to design an optimal way of scaling a high-performance API and our cloud system. API requests are distributed between several workers and AWS instances, giving them enough time to provide a response. Under peak load, we can create additional instances and avoid system crashes.

However, this method of scaling has several disadvantages you need to be aware of. The request queue inside the instance helps to manage API performance, but it shouldn't accumulate more requests than the instance can process within three minutes. Since the API connection lasts for three to four minutes, the API won't get responses to requests that take longer. That's why we need to launch a new instance during this time.

Also, you need to keep track of the minimum number of requests for your system and add two or more instances if you anticipate a load spike. You can automate this by writing a script that controls the minimum number of interfaces.

Contents:

Conclusion

Our experience implementing the scaling scheme we've described shows that there are several conditions you have to keep in mind to avoid performance issues:

1. Isolate processes using workers, which gives control the application over the performance of the instance.
2. Increase the lifetime of the API connection by the period necessary for the request from the queue to be executed in the worker.
3. If you use external services, remove restrictions on internet traffic.
4. Configure the load balancer to evenly distribute traffic between instances.

Taking into account future load and potential performance issues is an essential step in API development and the design of high-load cloud infrastructure. That's why at Apriorit, we build a prototype of a client's system and conduct load testing before we start development. It helps us figure out how to scale API performance in a high-load system, design the best possible scaling scheme for a particular project, and reduce the cost of cloud resources. Our cloud computing and virtualization development teams are ready to assist you with any cloud project.

Contact us

Conclusion

Have a questio

Ask our expe

Lidiia
R&D

Contents:

Conclusion

Have a questio

Ask our expe

Lidiia
R&D

Contents:

Conclusion

Have a questio

Ask our expe

Lidiia
R&D

Contents:

Tags:

CLOUD API

Share this article

Written by:

Andrii

Driver Development Team

Chief Information Security Officer

Ivan

Web Development Team

Development Coordinator

Contents:

Want more posts like this?
Sign up for our updates.

Your email

Cloud

I acknowledge that I have read and agree to the Privacy Policy and Terms & Conditions, and consent to receiving marketing communications from Apriorit.*

Sign me up

TECH INSIGHTS AND EXPERT TIPS

Have a questio

Ask our expe

Lidiia
R&D

Contents:

AI in FinTech: Trends, Use Cases, Challenges, and Best Practices

AI & ML BUSINESS FINTECH

Read more →

How to Use Data Visualization and AI-Powered Knowledge Graphs to Enhance Your Cybersecurity Product

AI & ML CYBERSECURITY CYBERSECURITY SOLUTIONS

Read more →

Have a questio

Ask our expe

Lidiia
R&D

Contents:

Have a questio

Ask our expe

Lidiia
R&D

Cybersecurity in FinTech Solutions: What to Watch Out For and How to Respond

CYBERSECURITY FINTECH

Read more →

Contents:

Have a questio

Ask our expe

Lidiia
R&D

Building a Secure AI Chatbot: Risk Mitigation and Best Practices

AI & ML CYBERSECURITY

Read more →

GET MORE POSTS LIKE THIS →

Contents:

TELL US ABOUT YOUR PROJECT

Email *

Phone

Name *

Company

Subject of your request

Give us more details on your project

ATTACH A FILE

Send an NDA

I acknowledge that I have read and agree to the Privacy Policy and Terms & Conditions, and consent to receiving marketing communications from Apriorit.*

SEND YOUR REQUEST →

Have a questio

Ask our expe

Lidiia
R&D

Contents:

Apriorit is a software engineering company with unique IT expertise. We help companies around the world tackle the toughest tech challenges in their journey to build secure and reliable software

+1 714-584-0295

info@apriorit.com

60 Kendrick St. Suite 201 Needham, MA 02494, USA

D-U-N-S number: 117063762

Company

How we work

Careers

Contacts

Blog

Custom Software Development

Dedicated teams

Secure SDLC

STAND WITH UKRAINE

Learn more how you can help >

System Programming & Extensions

Kernel & Driver

Network Management

Reverse engineering

SaaS Development

Mobile

Mobile App Development

Mobile Device and Application Management

API Integration

Research & Discovery

Technology-based Development

C/C++

Python

NodeJS

.NET

Angular

Cloud

Infrastructure Management

DevOps

Cloud Computing & Virtualization

Quality Assurance

Security Testing & Audits

Penetration Testing

Code Audit

UI/UX Design

AI & ML Development

Generative AI

AI Chatbots

Support & Maintenance

Blockchain Development

Security Audits

Smart Contact Audit

Liana
R&D

Contents: